INT3404 - IMAGE PROCESSING
Assignment 2
The Geometry of Vision:
3D World $\leftrightarrow$ 2D Image

Thuong Le Cong, Minh Kien Nguyen

September 2026

1 Introduction

In computer vision and graphics, understanding the relationship between 3D world scenes and their 2D projections in images is fundamental. This assignment explores the core principles of camera geometry, bridging the gap between real-world 3D structures and the flat images captured by cameras.

Why does this matter? Applications like autonomous driving, augmented reality, robotics, and 3D reconstruction (e.g., models from photos for films or virtual tours) rely on accurate 3D-2D mapping for scene interpretation, pose estimation, and environment rebuilding. Without it, self-driving cars couldn't navigate, and AR couldn't overlay virtual objects seamlessly.

Think of a camera as a "window" into the 3D world: light rays from objects form images but lose depth, causing ambiguities where multiple 3D points map to one 2D pixel. We'll model forward projection, reverse it using depth cues, calibrate for imperfections.

Learning goals. In this assignment, you should be able to:

- Implement forward projection ($3D \rightarrow 2D$) using homogeneous coordinates and camera matrices.
- Explain the single-view ambiguity (a pixel corresponds to a 3D ray) and resolve it using depth.
- Calibrate a single camera using a chessboard: estimate intrinsics K and lens distortion.

Starter code and Environment setup.

- **Starter code:** Use the provided Python starter code in `code_starter/`.
- **Environment setup:** Use Python ≥ 3.8 and install the dependencies from `requirements.txt`.
- **What to submit:**

– Your code: `code_starter/*.py`.
– A short PDF report answering the questions and including the required figures.

2 Forward Projection: 3D points → 2D pixels

Before we can understand images, we must understand how they are formed. Every pixel in a photo is the result of a 3D ray of light hitting a sensor. If we know where the camera is and where a 3D point is, we should be able to predict exactly which pixel that point will light up. This is the foundation of *rendering* and *pose estimation*.

From World to Pixel. The "Pinhole Camera Model" describes this transformation. It involves two distinct steps:

1. **Extrinsic (World → Camera:** We move from the global "World Frame" to the "Camera Frame". This is a rigid body transformation (Rotation R and Translation t).
2. **Intrinsics (Camera → Image):** We project the 3D point onto the 2D film plane and convert metric units (meters) into digital units (pixels). This is handled by the calibration matrix K.

Mathematically, the complete projection from a world point P_w to a homogeneous pixel $\tilde{p}$ is a sequence of linear matrix multiplications:

$$\underbrace{\begin{bmatrix}\tilde{u}\\ \tilde{v}\\ \tilde{w}\end{bmatrix}}_{\tilde{p}} = K \underbrace{\begin{bmatrix}R & t\end{bmatrix}}_{\text{Extrinsics}} \underbrace{\begin{bmatrix}X_w\\ Y_w\\ Z_w\\ 1\end{bmatrix}}_{P_w} \tag{1}$$

where $\tilde{p}$ is the point in *homogeneous coordinates*. Note that this equation gives us a 3-vector, but our image is a 2D plane. To get the actual pixel coordinates, we must perform the *perspective division* to normalize the vector. Explicitly, the perspective division steps to recover the Euclidean pixel coordinates $p = (u, v)$ are:

$$\begin{aligned}\text{Compute Homogeneous Vector:} \quad & \tilde{p} = [\tilde{u}, \tilde{v}, \tilde{w}]^T \\ \text{Perspective Division:} \quad & u = \frac{\tilde{u}}{\tilde{w}}, \quad v = \frac{\tilde{v}}{\tilde{w}}\end{aligned} \tag{2}$$

Here, $\tilde{w}$ corresponds to the depth of the point in the camera frame (Z_c).

2.1 Work to do (code)

Complete the implementation in `code_starter/part12.py`[1]:

1. **Homogeneous Helpers:** Implement `to_homogeneous` and `from_homogeneous`. Hint: Remember that dividing by the last component is essential for converting back to Euclidean space.

[1] You can add `--interactive` to visualize the input and output interactively.

2. **Pose Composition:** Implement `compose_extrinsic(R,t)` to build the 4×4 transformation matrix from $R(3 \times 3$ and $t(3 \times 1)$.

3. **The Projection Pipeline:** Implement `project_points` following Eq. 1 and 2:

   - Transform World ($X_w \rightarrow$) Camera (X_c) using Extrinsics.
   - Apply Intrinsics to get $[\tilde{u}, \tilde{v}, \tilde{w}]$.
   - Perform perspective division (divide by $\tilde{w}$) to get pixel coordinates.

4. **Visualize:** Render the projected points onto a blank image (e.g., 800×600) and save to: `figures/part1_projection.png`

2.2 Report (Part 1)

In your PDF report:

- **The loss of dimension:** Explain why Eq. 1 is not invertible. If you are given just a pixel (u, v) and the camera matrix K, why can you **not** know the exact 3D position (X, Y, Z) (explain in mathematically and geometrically)? Hint: A key property of homogeneous coordinates is *scale-invariance*.
- **Vanishing Points:** In generated image, parralel lines on the 3D appear to converge. Mathematically, which part of your code/formula caused this phenomenon?
- Include `figures/part1_projection.png`.

3 RGB-D Back-Projection: pixels + depth $\rightarrow$ 3D points

In Part 1, we established that forward projection collapses 3D space onto a 2D plane, causing the loss of depth information. However, modern sensors such as the Microsoft Kinect, iPhone FaceID, or LiDAR solve this by providing a **depth map**—an image where every pixel stores a distance value Z. With this additional information, we can invert the projection process. This allows us to "lift" a flat 2D image back into a 3D point cloud, which is the foundational step in 3D scanning and mapping.

The logic works as follows: a single pixel $p = (u, v)$ defines a ray in 3D space originating from the camera center (Fig. 1). Without depth, the 3D point could lie anywhere along this ray. The depth value Z_c resolves this ambiguity by determining exactly where along the ray the surface is located. Conceptually, unprojection is simply: Unprojection = Ray Direction $\times$ Depth.

3.1 Work to do (code)

The directory `data/rgbd/` contains the following input files:

- A pair consisting of an RGB image and a registered depth image. These share the same resolution and maintain per-pixel correspondence.
- A calibration text file defining the image dimensions, the intrinsic matrix K, and the distortion coefficients.

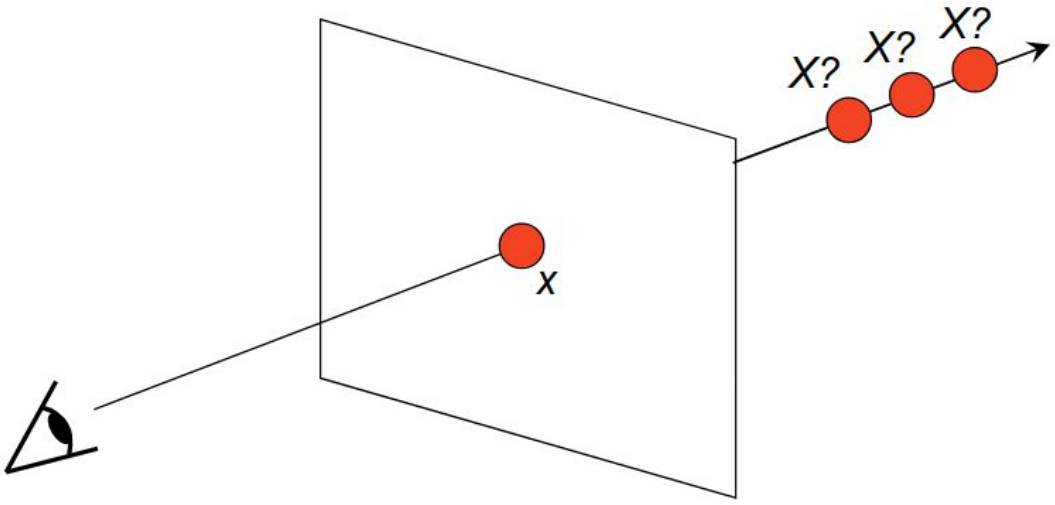


Figure 1: A single pixel $x = (u, v)$ defines a ray in space originating from the camera center. Without depth, the corresponding 3D point could be located anywhere on that ray. By adding a constraint, such as a known depth Z_c, we can determine the exact location (X) along the ray.

Dataset Specifications. This data is from the TUM RGB-D dataset[2]. The depth maps are stored as 16-bit unsigned integers. To convert the raw integer values into physical units (meters), you must apply the dataset scale factor of 5000:

$$Z_{meters} = \frac{\text{raw_depth_value}}{5000}$$

In `code_starter/part12.py`:

1. Assign the values from the provided calibration txt into the code: `RGBD_WIDTH`, `RGBD_HEIGHT`, the intrinsic matrix K, and `depth_scale` (e.g., 5000).
2. Load RGB and depth images. Convert depth to meters:

$$Z_c = D/\text{depth_scale}.$$

3. **Inverse Pipeline:** Implement `unproject_depth_image_to_pointcloud`.
   - Create a grid of pixel coordinates (u, v).
   - Apply the inverse formula derived above to compute X_c and Y_c using the provided Z.
   - Return `points_cam` $(N, 3)$ and `colors` $(N, 3)$.
4. Visualize the point cloud (subsample if needed) and save a screenshot as `part2_pointcloud.png`.

3.2 Report (Part 2)

In your PDF report:

[2] `https://cvg.cit.tum.de/data/datasets/rgbd-dataset/file_formats`

- **Unprojection derivation.** With $K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$ and $R = I$, $t = 0$ (camera frame $\equiv$ world frame, so $X_c = X_w$, etc.), derive X and Y, given depth Z.
- **Scale reasoning.** If depth is scaled by a constant ($Z' = sZ$) due to a wrong `depth_scale`, what changes in the point cloud and what does *not* change? Discuss the case $s = 2$.
- Include `figures/part2_pointcloud.png`.

4 Camera Calibration (Single camera + Chessboard): estimate K and distortion

So far, we assumed we magically knew the matrix K and that our lens was perfect. In reality, manufacturing a camera introduces imperfections:

- The sensor center is never exactly in the middle c_x, c_y.
- The lens bends light, causing straight lines to look curved (Radial Distortion).

Before a camera can be used for measurement (e.g., measuring the distance to a car in front of you), it must be **calibrated**. We do this by showing the camera a pattern of known geometry (a chessboard) and solving for the parameters that best align the observed pixels with the known 3D structure.

Calibration is an optimization problem. We try to find parameters (K,dist,R_i,t_i) that minimize the **reprojection Error**:

$$\text{Error} = \sum_i \|\text{observed_pixel}_i - \text{project}(\text{known_3D_point}_i)\|^2 \tag{3}$$

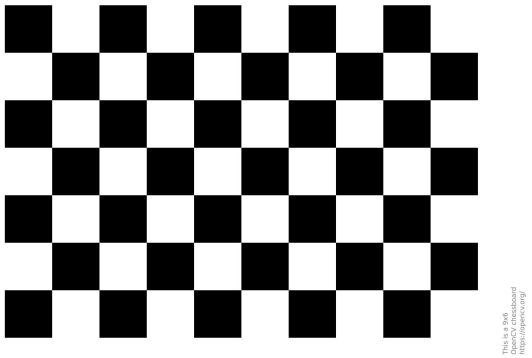
We want the difference between where we see the chessboard corner and where our math predicts it to be as close to zero as possible.

4.1 Work to do (code)

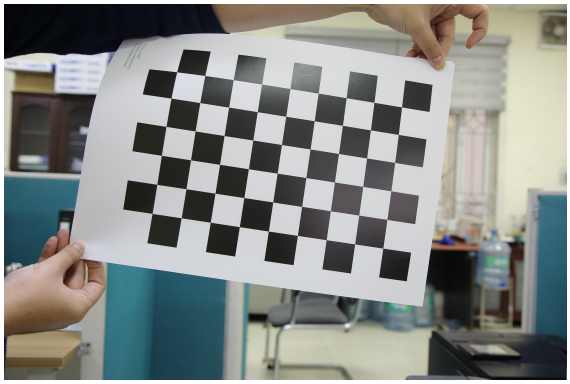
You have multiple images of a printed chessboard. Put all chessboard images here: `data/chessboard/`.

In `part3_calibrate_student.py`:

1. **Define Reality:** Implement `make_object_grid`. This generates the "ground truth" 3D coordinates of the chessboard corners (e.g., (0,0,0), (0.025,0,0), ...).
2. **Process Observations:** Use OpenCv to detect chessboard corners in the provided dataset (`cv2.findChessBoardCorners`).
3. **Solve:** Run `cv2.calibrateCamera`. This function performs the heavy optimization to solve for K and distortion coefficients.
4. **Analyze:** Compute the average reprojection error to verify your calibration quality.



(a) Chessboard pattern used in this assignment: **(9,6)=9 × 6 inner corners**.



(b) Example photo from the provided dataset (many views with different tilts/distances).

Figure 2: Calibration data: the chessboard pattern definition (left) and a sample captured image (right).

4.2 Report (Part 3)

In your PDF report:

- **Interpretation:** Report your estimated K matrix. Does the principal point (c_x, c_y) match the image center (width/2, height/2)? How far off is it?
- **The need for angles:** Why is it necessary to capture the chessboard from multiple different angles and distances? What would happen to the solver if you only took 10 images where the chessboard was perfectly parallel to the camera?
- Insert `figures/part3_corners.png`.
- Report K, distortion vector, and **mean reprojection error** (pixels).

5 Instructions for Completing the Assignment

- The starter code is available on the UET Portal.
- The report template is available at `https://www.overleaf.com/read/cgdgnkbbbyjz#b9a381`.
- After completing the assignment, you need to compress all the finalized source code, resulting images, and the PDF report into a `.zip` file, name it according to the format `FullName_StudentID`, and submit it on the UET Portal.